Inventory Management System

Introduction

The Inventory Management System helps a user maintain a simple inventory of products. Through a menu, the user can add new products, view all saved products, search by product ID, update details, delete records, and generate a stock summary. The application stores records in a text file so that data can be loaded the next time the program starts.

Objective

• Develop a menu-driven Java application that performs common inventory operations.

• Apply object-oriented programming concepts such as classes, objects, inheritance, abstraction, interfaces, and encapsulation.

• Use Java collections to store and find product records efficiently.

• Save and retrieve inventory data using file handling.

• Handle invalid input and demonstrate a basic multithreading task.

Features

• Add Record: Enter a unique product ID, name, quantity, and price.

• View All Records: Display every product and its calculated stock status.

• Search Record: Find a product using its ID.

• Update Record: Change a product name, quantity, price, or all details.

• Delete Record: Remove a product from the inventory.

• Stock Report: Show total products and counts for available, low-stock, and out-of-stock items.

• Input Validation: Reject invalid numeric values, negative quantities/prices, empty names, and duplicate IDs.

• File Storage: Load records from and save records to inventory.txt.

• Exit: Close the application through the menu.

Concepts Used

Concept	Where it is used
Classes and Objects	The Product class defines product data; each product is an object.
Constructors and this	Constructors initialize product details; this refers to the current object.
Static members	Shared collections and utility methods belong to the outer class.
Inheritance	Product extends the abstract Item class.

Polymorphism	Product provides its own display() and getStockStatus() implementations.
Encapsulation	Quantity and price are private and accessed through getters/setters.
Abstraction and Interface	Item is abstract; Stock defines getStockStatus().
Exception Handling	try-catch handles invalid input and file errors.
Collections	ArrayList stores products; HashSet tracks unique IDs; HashMap supports ID lookup.
File Handling	BufferedReader and FileWriter read/write inventory.txt.
Multithreading	A separate Thread generates the stock report.
Switch-Case	The menu routes the user's choice to the correct operation.

Project Structure

```
InventoryManagementSystem.java
└── InventoryManagementSystem
    ├── Scanner and collections
    │   ├── ArrayList<Product>
    │   ├── HashSet<Integer>
    │   └── HashMap<Integer, Product>
    ├── Stock (interface)
    │   └── getStockStatus()
    ├── Item (abstract class)
    │   ├── productId
    │   ├── productName
    │   └── display()
    ├── Product (extends Item, implements Stock)
    │   ├── quantity
    │   ├── price
    │   └── getters, setters, stock status
    ├── Input helper methods
    ├── Add / View / Search / Update / Delete
    ├── saveData() / loadData()
    ├── stockReport() [Thread]
    └── main() [switch-case menu]
```

Code Screenshots

```
static class Product extends Item implements Stock
    {
        private int quantity;
        private double price;

        Product(int productId, String productName, int quantity, double price)
        {
            super(productId, productName);
            this.quantity = quantity;
```

```
        this.price = price;
    }

    public int getProductId()
    {
        return productId;
    }

    public String getProductName()
    {
        return productName;
    }

    public int getQuantity()
    {
        return quantity;
    }

    public double getPrice()
    {
        return price;
    }

    public void setProductName(String productName)
    {
        this.productName = productName;
    }

    public void setQuantity(int quantity)
    {
        this.quantity = quantity;
    }

    public void setPrice(double price)
    {
        this.price = price;
    }

    public String getStockStatus()
    {
        if (quantity == 0)
            return "Out of Stock";
        else if (quantity <= 5)
            return "Low Stock";
        else
            return "Available";
    }
```

```
static void addProduct()
    {
        int id = readInt("Enter Product ID: ");

        if (id <= 0)
        {
            System.out.println("Product ID must be positive.");
            return;
        }

        if (productIds.contains(id))
        {
            System.out.println("Product ID already exists!");
            return;
        }

        String name = readString("Enter Product Name: ");
        int quantity = readInt("Enter Quantity: ");
        double price = readDouble("Enter Price: ");

        if (quantity < 0 || price < 0)
        {
            System.out.println("Quantity and price cannot be negative.");
            return;
        }

        Product p = new Product(id, name, quantity, price);

        products.add(p);
        productIds.add(id);
        productMap.put(id, p);

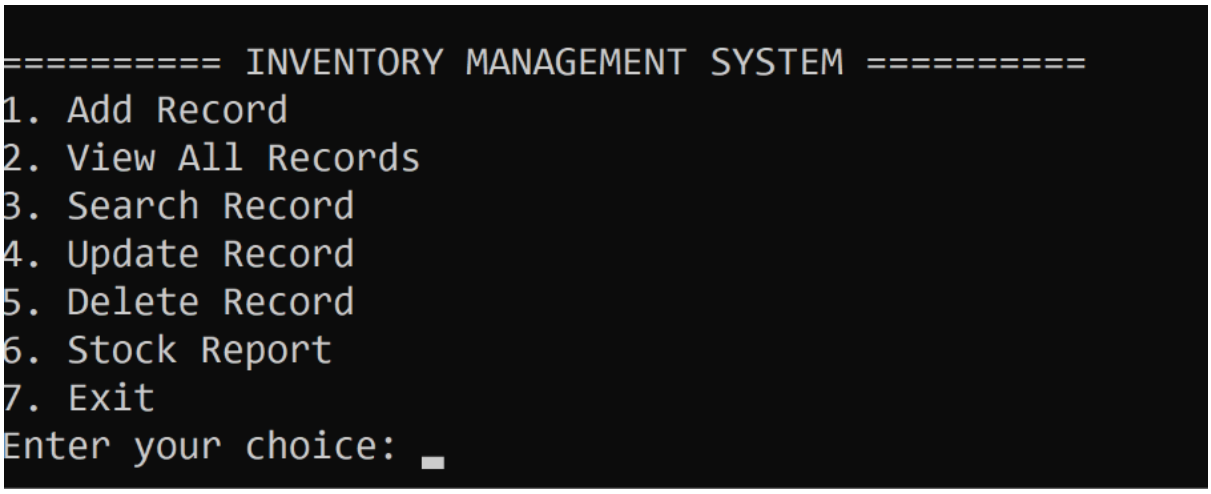
        saveData();

        System.out.println("Product added successfully!");
    }
```

```
static void saveData()
    {
        try (PrintWriter pw = new PrintWriter(new FileWriter(FILE_NAME)))
        {
            for (Product p : products)
            {
```

```
                pw.println(
                    p.getProductId() + "|" +
                    p.getProductName() + "|" +
                    p.getQuantity() + "|" +
                    p.getPrice()
                );
            }
        }
        catch (IOException e)
        {
            System.out.println("Error saving inventory: " + e.getMessage());
        }
    }
```

Output Screenshots



```
========== INVENTORY MANAGEMENT SYSTEM ==========
1. Add Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. Stock Report
7. Exit
Enter your choice: 1
Enter Product ID: 101
Enter Product Name: washing machine
Enter Quantity: 10
Enter Price: 200000
Product added successfully!
```

```
========== INVENTORY MANAGEMENT SYSTEM ==========
1. Add Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. Stock Report
7. Exit
Enter your choice: 3
Enter Product ID to search: 101
-----------------------------------------
Product ID   : 101
Product Name : washing machine
Quantity     : 10
Price        : 200000.0
Stock Status : Available
-----------------------------------------
```

```
========== INVENTORY MANAGEMENT SYSTEM ==========
1. Add Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. Stock Report
7. Exit
Enter your choice: 4
Enter Product ID to update: 101
1. Update Product Name
2. Update Quantity
3. Update Price
4. Update All Details
Enter your choice: 2
Enter New Quantity: 8
Product updated successfully!
```

```
========== INVENTORY MANAGEMENT SYSTEM ==========
1. Add Record
2. View All Records
3. Search Record
4. Update Record
5. Delete Record
6. Stock Report
7. Exit
Enter your choice: 6

Generating Stock Report...

========== STOCK REPORT ==========
Total Products : 1
Available      : 1
Low Stock      : 0
Out of Stock   : 0
==================================
```

Challenges Faced

- Preventing duplicate product IDs: A HashSet is used to check whether an ID already exists before adding a product.
- Finding products quickly: A HashMap stores products by ID, allowing direct lookup for search, update, and delete operations.
- Handling invalid input: Numeric input helpers use exception handling to keep asking until a valid integer or number is entered.

- Keeping data after program exit: File handling writes records to inventory.txt and loads them when the application starts.
- Maintaining correct stock status: Stock status is calculated from the current quantity, so it changes automatically after a quantity update.
- Demonstrating multithreading: The stock report is run inside a separate Thread, and join() ensures the report completes before the program continues.

Conclusion

The Inventory Management System demonstrates how Java can be used to build a practical, menu-driven application. During development, the project applies object-oriented programming, collections, file handling, exception handling, interfaces, abstract classes, and basic multithreading. It also provides hands-on practice in validating data and organizing program operations with switch-case. The project can be extended in the future with a graphical user interface, database connectivity, user login, and transaction history.
